My Bank – An In-house Banking Application

Project Report Submitted in completion of
Java External Training for

OSS Trainees (July'21)
of
Yamaha Motor Solutions India Limited

Submitted by

Vinayak Kuradia: (VE00YM264)

Under the Supervision of

Mr. Rajeev Gupta
Ext. Trainer

Sept, 2021

Contents

1. **Introduction** **1**
2. **Technical Description** 2
3. **Design & Working** 4
4. **Conclusion & Future work** 15

References…… 16

Chapter 1

Introduction

In a growing country like India, banking industry is also continuously growing. From the introduction of technology and internet, the world of not only banking sectors but all financial sectors have changed drastically. Banks are responsible for daily transactions of its customer, which may be related to cash, cheques, loans, EMIs or many more. But to perform these transactions in today's date, banks use computer systems with support of well managed and organized databases. The software applications used in banks also require the basic functionalities like storing information about new customer, updating, and deleting it as per requirement. As previously mentioned, banks deal with thousands of transactions each day but the most important aspect of these transactions is that they must be accurate, and consistent at the same time, but also fast... because the time is precious in today's world.

- **Objective**

The objective of this project is to implement a web-based application for the internal uses of any bank, which includes day to day operation of creating customer and account records, making transactions like deposit, withdraw and transfer of money. The project has two main users of the application, first, the clerk who can perform all the basic tasks like creating records and updating them and to perform transactions for the bank customers. Second, the manager who have administrative powers in application like deleting records, approving transactions above a certain limit and much more. Along with these basic requirements the application must also have authentication system which requires its users to input username and password for authentication, to use the system or say perform any kind of operation.

Chapter 2

Technical Description

2.1 Technologies Used

- **Java 8**

  Java is a high-level, class-based, object-oriented programming language that is designed to have as few implementation dependencies as possible.

- **Spring Boot**

  The Spring Framework is an application framework and inversion of control container for the Java platform.

Fig. 2.1: Java and Spring

- **Spring Security**

  Spring Security is a Java/Java EE framework that provides authentication, authorization, and other security features for enterprise applications

- **Spring Data JPA**

This module makes it easy to easily implement JPA based repositories. It deals with enhanced support for JPA based data access layers.

- **Maven**

  Maven is a build automation tool used primarily for Java projects.

- **Hibernate**

  It is an object–relational mapping tool for the Java programming language. It provides a framework for mapping an object-oriented domain model to a relational database.

- **Jakarta Server Pages** (JSP)

  Jakarta Server Pages is a collection of technologies that helps software developers create dynamically generated web pages based on HTML, or other document types

- **Bootstrap**

  Bootstrap is a CSS Framework for developing responsive websites. It is also used to make websites in lesser time with its available web elements.

- **MySQL**

  MySQL is a relational database management system based on SQL – Structured Query Language.

Chapter 3

Design and Working

In this chapter we'll discuss about design and working aspect of the project.

3.1 Object Oriented Analysis and Design (OOAD) of Project

It is a technical approach for analysing and designing an application by applying object-oriented programming, as well as using visual modeling throughout the software development process.

- **Use Case Diagram:**

  The diagram tries to graphically describe the interaction of users with the system and possible operations and interactions possible within the system.

  In this diagram, as mentioned in previous section, there are two users 1. Clerk, 2. Manager. Manager holds the administrative power while clerk has ability to perform day to day operations an when the operation needs approval the approval can be given by manager.

  The diagram has four sections namely **Authentication** which refers to the login logout functionality, next, **Transactions** refers to the transaction records of operations processed by the application, next **Operations** refers to day to day operation of depositing, withdrawing and transferring funds from one account to another.

  Next, **Customer** and **Saving Account** refers to records of customer of the bank and their respective savings account in the bank.

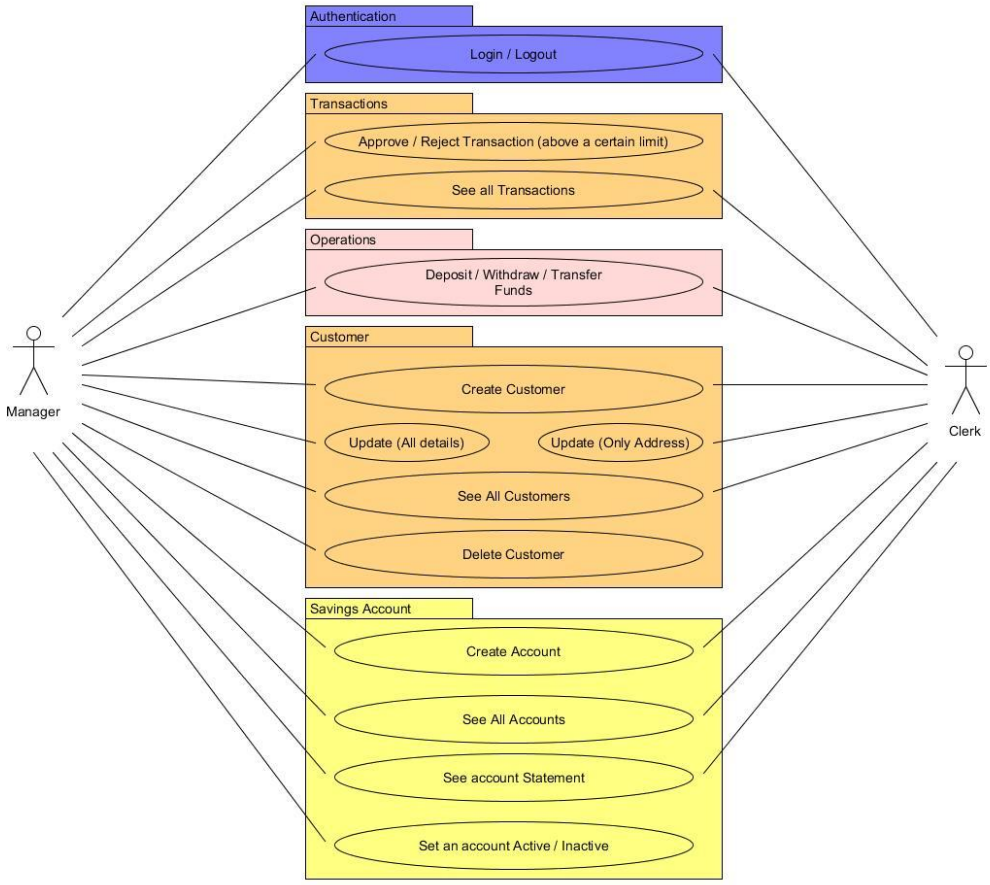


Fig. 3.1: Use Case Diagram of My Bank Application

- **Class Diagram:**

Class diagram is a static diagram. It represents the static view of an application. Class diagram is not only used for visualizing, describing, and documenting different aspects of a system but also for constructing executable code of the software application.

The following class diagrams depicts multiple classes implemented in the application like: **Account** (Abstract), **Savings Account**, **Transaction**, **Customer**, **CustomerData**, **Name**, **Address**. Enums are also used in the projects, namely: **AccountType**, **Sex**, **TransactionType**, **TransactionStatus**, **State** and **District**.

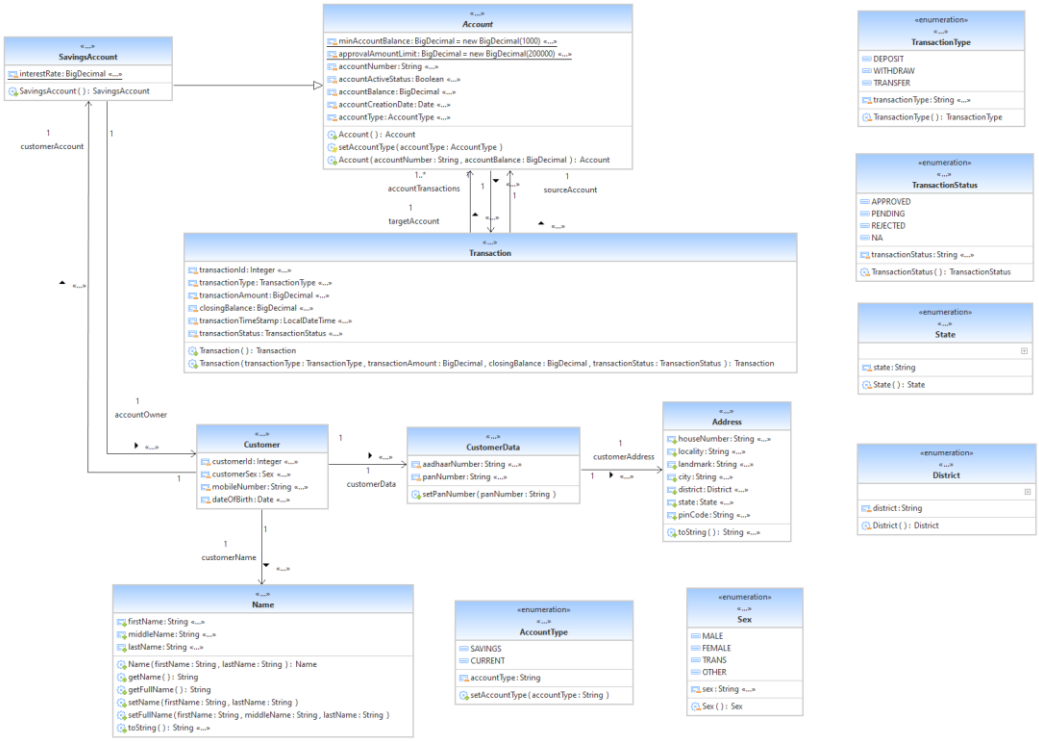


Fig. 3.2: Class Diagram of My Bank Application

3.2 Database Design

An entity–relationship model describes interrelated things of interest in a specific domain of knowledge. A basic ER model is composed of entity types and specifies relationships that can exist between entities.

Even though the work of creating tables and inserting data is handled by hibernate we can always modify the database as per our requirement, the following diagram describes the implemented database design of tables. In this case we're only having one to one relationship between different entities. So let's have a look on the diagram and try to understand it.

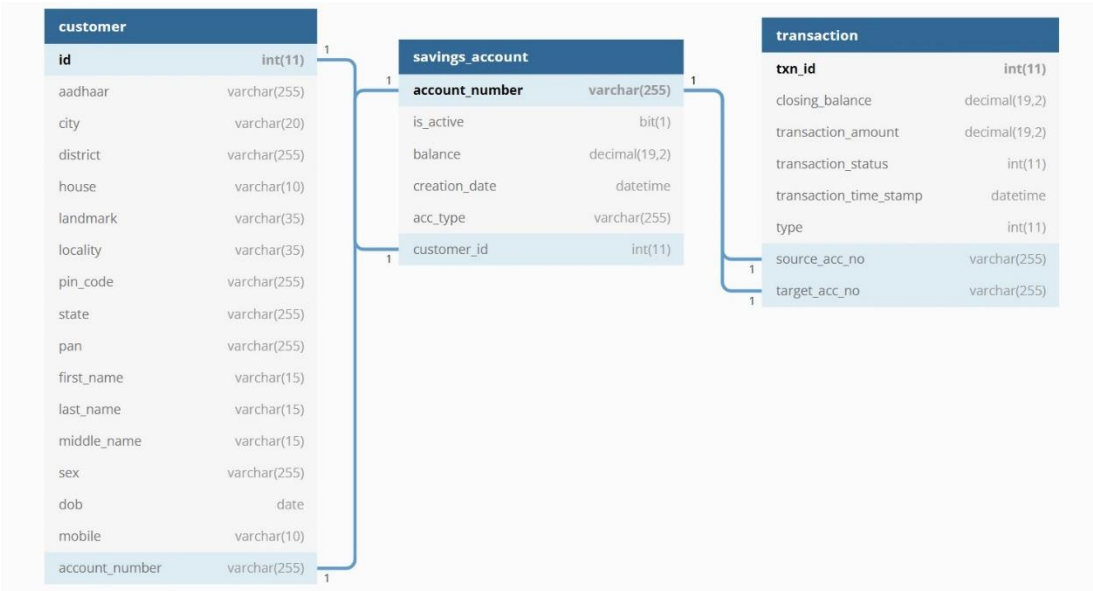


Fig. 3.3: ER Diagram of My Bank Application

There are 3 tables, each table is containing a primary key which uniquely identifies tuples. Tables are also containing foreign keys for referencing tuples/records in other tables.

3.3 Coding Screenshots

Below are the screenshots of implemented code and project structure. Project structure is divided into several packages as per three tier architecture and according to some best practices and design patterns.

Entities are divided into three separate sub packages, operational entities are the main entities that are used in applications while non-operational entities are addon to the main (operational entities) and finally, data transfer object (DTO) entities are used for validation purposes, they don't serve much purpose otherwise.

There are three main layers of web arch., those are persistence, controller, and service layer. All three are implemented while service layer implements façade design pattern to reduce complexity of uses within system and as a "best practice".

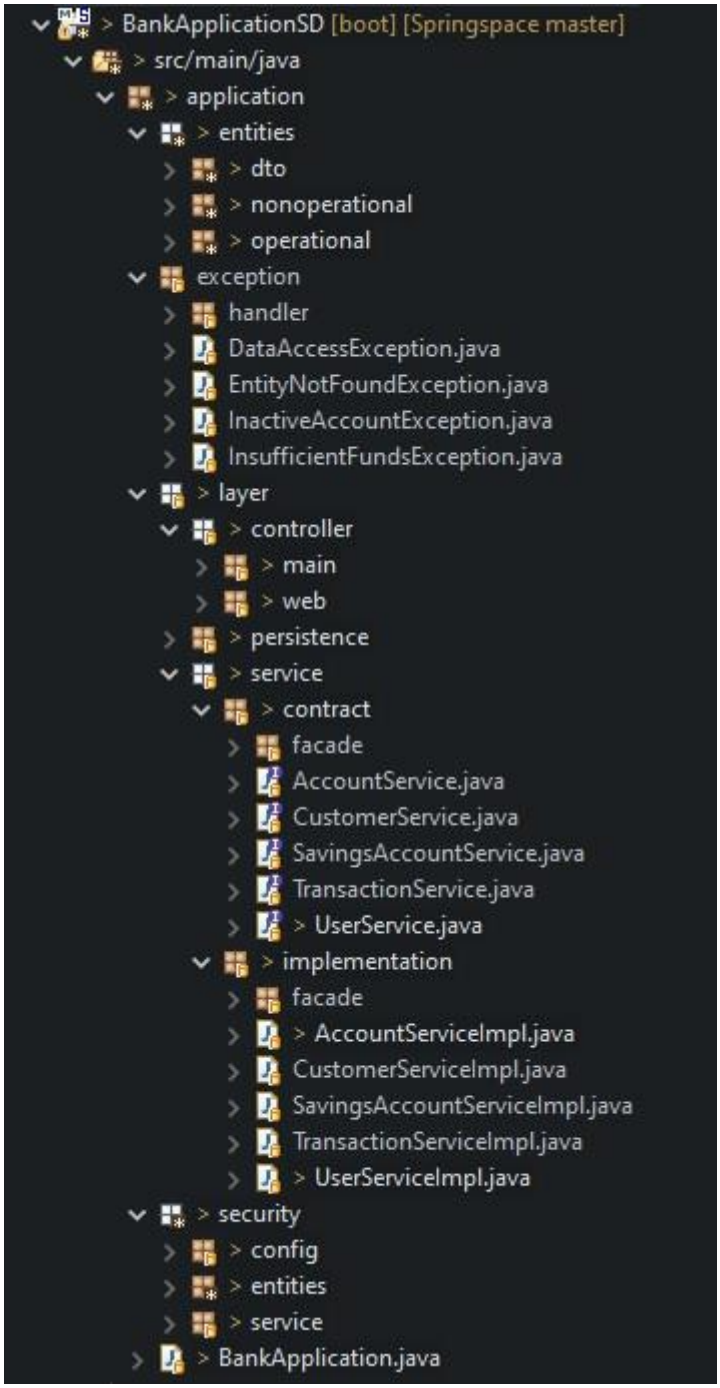


Fig. 3.4: Project Structure

Exception package contains basic exceptions needed in application and contains the handler for exceptions.

While security package contains all the required security classes like web security configuration file, security user entity and custom user detail service for authentication using database and it also implements BCryptPasswordEncoder to preserve confidentiality of user passwords and security purposes.

Below are some of the screenshots of code implementations:

```java
Account.java
package application.entities.operational;

import java.math.BigDecimal;

@Getter
@Setter
@Entity
@Inheritance(strategy = InheritanceType.TABLE_PER_CLASS)
public abstract class Account {

    // Static Part
    @Getter @Setter
    private static BigDecimal minAccountBalance=new BigDecimal(1000);

    @Getter @Setter
    private static BigDecimal approvalAmountLimit=new BigDecimal(200000);

    // Instance Part
    @Id @GeneratedValue(generator="system-uuid")
    @GenericGenerator(name="system-uuid", strategy = "uuid")
    private String accountNumber;

    @Column(name = "isActive")
    @NotNull(message = "Account active status is mandatory")
    private Boolean accountActiveStatus;

    @Column(name = "balance")
    @NotNull(message = "Account balance is mandatory")
    @DecimalMin(value = "0.00", message = "Balance cannot be less than 0")
    private BigDecimal accountBalance;

    @Column(name = "creationDate")
    @NotNull(message = "Account creation date is mandatory")
    private Date accountCreationDate;

    @Column(name = "accType")
    @Enumerated(EnumType.STRING)
    @NotNull(message = "Account type is mandatory")
    private AccountType accountType;

    @OneToMany
    @ToString.Exclude
    private List<Transaction> accountTransactions;

    public Account() {
        this.accountActiveStatus = true;
        this.accountCreationDate = new Date();
    }
```

```java
AccountRepository.java   CustomerRepository.java   SavingsAccountRepository.java   TransactionRepository.java
package application.layer.persistence;

import java.util.List;

@Repository
public interface TransactionRepository extends JpaRepository<Transaction, Integer> {
    @Query("select t.transactionId from Transaction t")
    public List<Integer> getAllTransactionsIds();

    @Query("from Transaction where transactionType= :transactionType")
    public List<Transaction> getAllTransactionsByType(@Param("transactionType") TransactionType transactionType);

    @Query("from Transaction where transactionStatus= :transactionStatus")
    public List<Transaction> getAllTransactionsByStatus(@Param("transactionStatus") TransactionStatus transactionStatus);

    @Query("from Transaction where sourceAccount= :account or targetAccount= :account")
    public List<Transaction> getAllTransactionsByAccount(@Param("account") Account account);
}
```

SavingsAccountController.java | SiteController.java | TransactionController.java | CustomerController.java | AccountController.java

```java
package application.layer.controller.web;

import java.text.SimpleDateFormat;

@Controller("customer")
@RequestMapping(path = "/customer/")
public class CustomerController {

    private BankFacade bankFacade;

    List<Sex> sexTypes = new ArrayList<Sex>(Arrays.asList(Sex.values()));
    List<District> districts = new ArrayList<District>(Arrays.asList(District.values()));
    List<State> states = new ArrayList<State>(Arrays.asList(State.values()));

    @Autowired
    public CustomerController(BankFacade bankFacade) {
        this.bankFacade = bankFacade;
    }

    @InitBinder
    public void initBinder(WebDataBinder binder) {
        binder.registerCustomEditor(Date.class, new CustomDateEditor(new SimpleDateFormat("yyyy-MM-dd"), true));
    }

    @GetMapping(path = "/addCustomer")
    public String addCustomer(ModelMap map) {
        map.addAttribute("sexTypes", sexTypes);
        map.addAttribute("districts", districts);
        map.addAttribute("states", states);
        map.addAttribute("customer", new Customer());
        return "customer/addCustomer";
    }

    @PostMapping(path = "/addCustomer")
    public String addCustomer(@Valid @ModelAttribute(name = "customer") Customer customer, BindingResult br, ModelMap map) {
        if (br.hasErrors()) {
            map.addAttribute("sexTypes", sexTypes);
            map.addAttribute("districts", districts);
            map.addAttribute("states", states);
            return "customer/addCustomer";
        } else {
            bankFacade.addCustomer(customer);
            return "redirect:/customer/";
        }
    }

    @GetMapping(path = "")
    public String showAllCustomers(ModelMap map) {
```

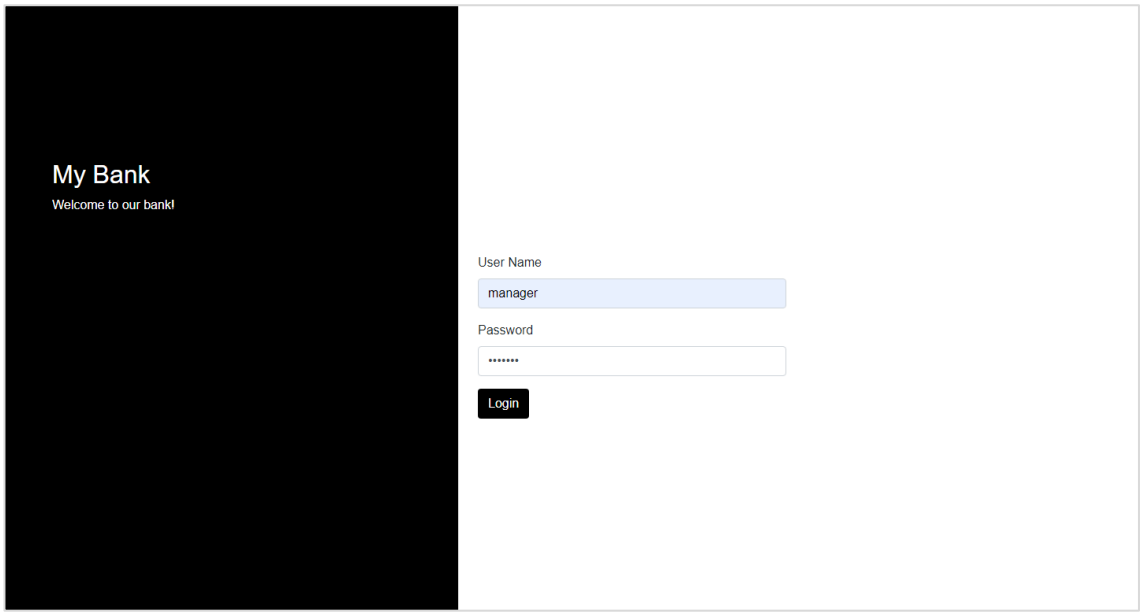
*WebSecConfig.java

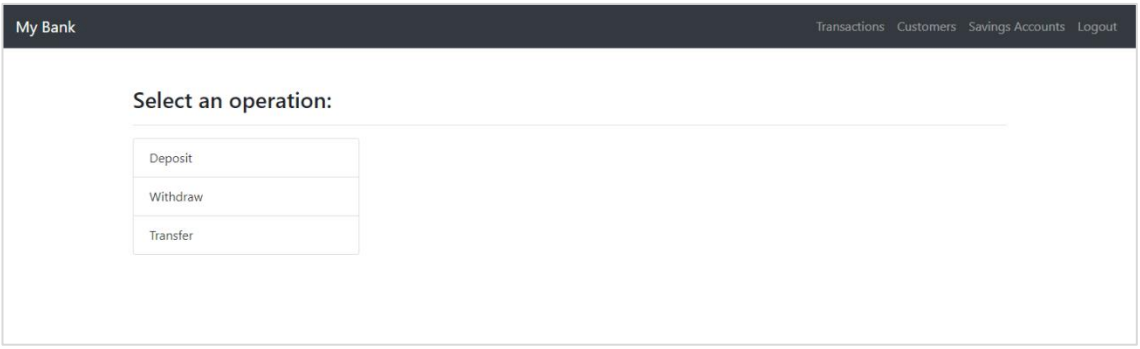
```java
package application.security.config;

import org.springframework.beans.factory.annotation.Autowired;

@EnableWebSecurity
public class WebSecConfig extends WebSecurityConfigurerAdapter {

    private UserDetailsService userDetailsService;

    @Autowired
    public WebSecConfig(UserDetailsService userDetailsService) {
        this.userDetailsService = userDetailsService;
    }

    @Override
    protected void configure(AuthenticationManagerBuilder auth) throws Exception {
        auth.userDetailsService(userDetailsService);
    }

    @Bean
    public PasswordEncoder getPasswordEncoder() {
        return new BCryptPasswordEncoder();
    }

    @Override
    protected void configure(HttpSecurity http) throws Exception {
        http
            .csrf().disable()
            .authorizeRequests()
            .antMatchers("/**").hasAnyRole(AuthRole.ADMIN.getPlainRole(), AuthRole.USER.getPlainRole())
            .antMatchers("/**/delete*/**", "/**/approve*/", "/**/reject*/").hasAnyRole(AuthRole.ADMIN.getPlainRole())
                .antMatchers("/").permitAll()
                .antMatchers("error*").permitAll()
            .and()
                .formLogin()
                .loginPage("/")
                .loginProcessingUrl("/login")
                .usernameParameter("username")
                .passwordParameter("password")
                .defaultSuccessUrl("/home")
                .permitAll()
            .and()
                .logout()
                .logoutUrl("/logout")
                .logoutSuccessUrl("/")
                .invalidateHttpSession(true)
                .deleteCookies("JSESSIONID")
                .clearAuthentication(true)
```

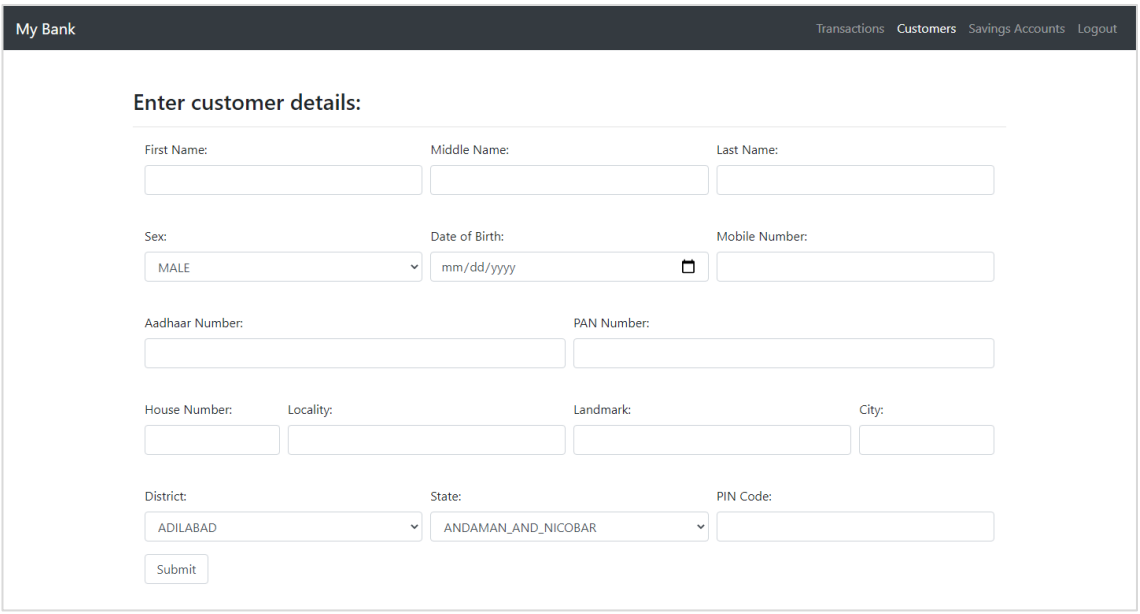
```java
BankFacadeImpl.java
package application.layer.service.implementation.facade;

import java.math.BigDecimal;

@Repository
@Transactional
public class BankFacadeImpl implements BankFacade {

    private AccountService accountService;
    private CustomerService customerService;
    private SavingsAccountService savingsAccountService;
    private TransactionService transactionService;

    @Autowired
    public BankFacadeImpl(AccountService accountService, CustomerService customerService, SavingsAccountService savingsAccountService, TransactionServ
        this.accountService = accountService;
        this.customerService = customerService;
        this.savingsAccountService = savingsAccountService;
        this.transactionService = transactionService;
    }

    // Accounts

    @Override
    public BigDecimal withdraw(String accountNumber, BigDecimal amount) {
        return accountService.withdraw(accountNumber, amount);
    }

    @Override
    public BigDecimal deposit(String accountNumber, BigDecimal amount) {
        return accountService.deposit(accountNumber, amount);
    }

    @Override
    public BigDecimal transfer(String sourceAccountNumber, String targetAccountNumber, BigDecimal amount) {
        return accountService.transfer(sourceAccountNumber, targetAccountNumber, amount);
    }

    @Override
    public void changeAccountActiveStatus(String accountNumber) {
        accountService.changeAccountActiveStatus(accountNumber);
    }

    // Customers

    @Override
    public List<Integer> getAllEligibleCustomersIds() {
        return customerService.getAllEligibleCustomersIds();
```

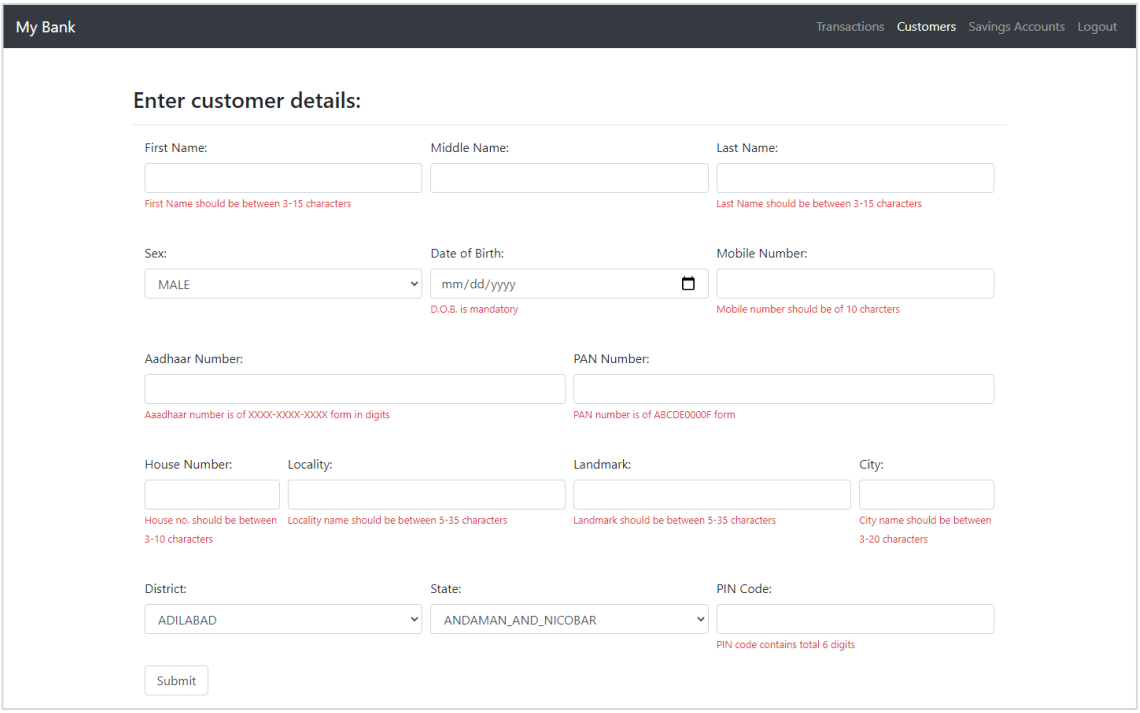
3.4 Working Screenshots

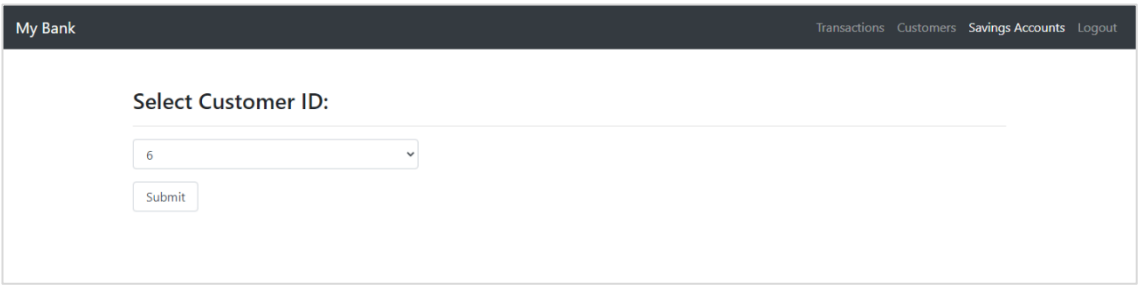
Below are the working screenshots of the application:

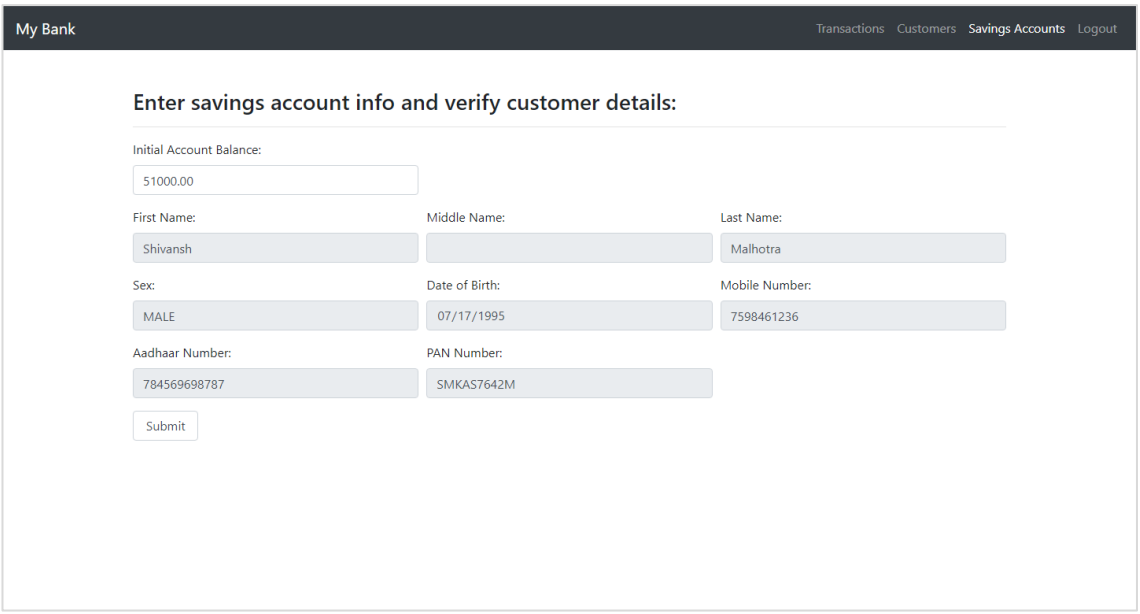



My Bank
Transactions
Customers
Savings Accounts
Logout
Select an operation:
Deposit
Withdraw
Transfer


My Bank
Transactions
Customers
Savings Accounts
Logout
Enter customer details:
First Name:
Middle Name:
Last Name:
Sex:
MALE
Date of Birth:
mm/dd/yyyy
Mobile Number:
Aadhaar Number:
PAN Number:
House Number:
Locality:
Landmark:
City:
District:
ADILABAD
State:
ANDAMAN_AND_NICOBAR
PIN Code:
Submit


My Bank
Transactions
Customers
Savings Accounts
Logout
Enter customer details:
First Name:
Middle Name:
Last Name:
First Name should be between 3-15 characters
Last Name should be between 3-15 characters
Sex:
MALE
Date of Birth:
mm/dd/yyyy
D.O.B. is mandatory
Mobile Number:
Mobile number should be of 10 charcters
Aadhaar Number:
Aaadhaar number is of XXXX-XXXX-XXXX form in digits
PAN Number:
PAN number is of ABCDE0000F form
House Number:
Locality:
Landmark:
City:
House no. should be between 3-10 characters
Locality name should be between 5-35 characters
Landmark should be between 5-35 characters
City name should be between 3-20 characters
District:
ADILABAD
State:
ANDAMAN_AND_NICOBAR
PIN Code:
PIN code contains total 6 digits
Submit


My Bank
Transactions
Customers
Savings Accounts
Logout
Select Customer ID:
6
Submit


My Bank
Transactions
Customers
Savings Accounts
Logout
Enter savings account info and verify customer details:
Initial Account Balance:
51000.00
First Name:
Shivansh
Middle Name:
Last Name:
Malhotra
Sex:
MALE
Date of Birth:
07/17/1995
Mobile Number:
7598461236
Aadhaar Number:
784569698787
PAN Number:
SMKAS7642M
Submit

My Bank Transactions Customers Savings Accounts Logout

All customers:

Id	Name	Sex	Mobile No.	D.O.B.	Aadhaar No.	PAN No.	Address	Operations
1	Krishnan Maheshwari	MALE	7849532603	07-09-1998	123412341234	ABCDE3210N	77B Unknown Lankdmark 1 Agra AGRA UTTAR_PRADESH 749125	Update Delete
2	Radhika Joshi	MALE	1023456789	08-01-1996	545469698767	AEDED3124N	123 Locality 1 Lankdmark 4 Akola AKOLA MAHARASHTRA 447823	Update Delete
3	Rajan Sharma	MALE	4512354891	05-01-1994	545469698767	ABCDE3210N	77B Some Locality Lankdmark 1 Agra ALIRAJPUR UTTAR_PRADESH 749125	Update Delete
5	Rudra Pratap Singh	MALE	4568712549	07-01-2003	372805727652	RFGTS8459J	23E Lawyer colony Near court My city ALMORA JHARKHAND 487512	Update Delete

Add Customer

My Bank Transactions Customers Savings Accounts Logout

All savings account:

accountNumber	accountActiveStatus	accountBalance	accountCreationDate	Customer Id	Account Owner	Mobile No.	Transactions
402881857c0dd8a0017c0dd914f80000	true change	395307174.00	22-09-2021	1	Krishnan Maheshwari	7849532603	Transactions
402881857c0e79f0017c0e7c786f0000	true change	454109128.00	22-09-2021	2	Radhika Joshi	1023456789	Transactions
402881857c12a7c4017c12a8c6f30000	true change	45892440710.11	23-09-2021	3	Rajan Sharma	4512354891	Transactions
402881857c133a69017c133d46ce0000	true change	458701907946.00	23-09-2021	5	Rudra Pratap Singh	4568712549	Transactions

Add Savings Account

My Bank Transactions Customers Savings Accounts Logout

All transactions:

TxnId	Source Account	Target Account	Type	Txn Amount	ClosingBalance	TimeStamp	Status	Action
1	402881857c0dd8a0017c0dd914f80000	402881857c0e79f0017c0e7c786f0000	TRANSFER	210000.00	399790000.00	2021-09-22T22:43:18	REJECTED	NA
2	402881857c0dd8a0017c0dd914f80000	402881857c0e79f0017c0e7c786f0000	TRANSFER	4578128.00	395211872.00	2021-09-22T23:32:59	APPROVED	NA
3		402881857c12a7c4017c12a8c6f30000	DEPOSIT	45893486454.11	45893551454.11	2021-09-24T17:57:09	NA	NA
4		402881857c133a69017c133d46ce0000	DEPOSIT	458701472034.00	458701947946.00	2021-09-24T17:57:26	NA	NA
5	402881857c12a7c4017c12a8c6f30000	402881857c0e79f0017c0e7c786f0000	TRANSFER	41000.00	45893510454.11	2021-09-24T17:58:15	NA	NA
6	402881857c0dd8a0017c0dd914f80000	402881857c0e79f0017c0e7c786f0000	TRANSFER	350000.00	394861872.00	2021-09-24T18:01:29	APPROVED	NA
7	402881857c0dd8a0017c0dd914f80000		WITHDRAW	45698.00	394816174.00	2021-09-24T18:02:35	NA	NA
8	402881857c12a7c4017c12a8c6f30000		WITHDRAW	518744.00	45892991710.11	2021-09-24T18:02:45	NA	NA
9	402881857c133a69017c133d46ce0000		WITHDRAW	40000.00	458701907946.00	2021-09-24T18:02:57	NA	NA
10	402881857c0dd8a0017c0dd914f80000		WITHDRAW	60000.00	394756174.00	2021-09-24T18:03:06	NA	NA
11	402881857c12a7c4017c12a8c6f30000	402881857c0dd8a0017c0dd914f80000	TRANSFER	551000.00	45892440710.11	2021-09-24T18:14:53	APPROVED	NA

.

Chapter 4

Conclusion & Future Work

The My Bank, web application is implemented using some of the best practices of object-orientation and design pattern. Even though the project implements some best practices, it's not enough for an application to stand in real world use cases.

For the application to be used in a real world, it needs to become robust in all real-world scenarios where the application might fail currently like in maintaining consistency during power outs and also, it needs to go through rigorous testing process to be qualified as a production level application. Apart from that performance measures needs to be taken, the code can be refactored and Java 8 features such as streams, parallel processing, also spring framework's transaction management can also be used to improve transaction's performance and to reduce the database hits.

4.1 Future Work

For future work, improvements can be done in user interface. Rather than JSP pages, modern front-end frameworks like Angular or React can be used to distribute the processing between client and server systems. Which would also require the implementation of the Spring REST module for interaction between front end and back end. Also considering growth and scalability factors, microservice is the future. Luckily spring also provides support for microservices via it's microservice module, which can provide sky heights to the scalability of the applications while also introducing some complexities in the system but it's a trade-off that we must manage according to the situation of present.

References

[1] Java (Programming language) - Wikipedia
[https://en.wikipedia.org/wiki/Java_(programming_language)]

[2] Spring Framework - Wikipedia
[https://en.wikipedia.org/wiki/Spring_Framework]

[3] Spring Data JPA
[https://spring.io/projects/spring-data-jpa]

[4] Introduction – Bootstrap v4.6
[https://getbootstrap.com/docs/4.6/getting-started/introduction/]